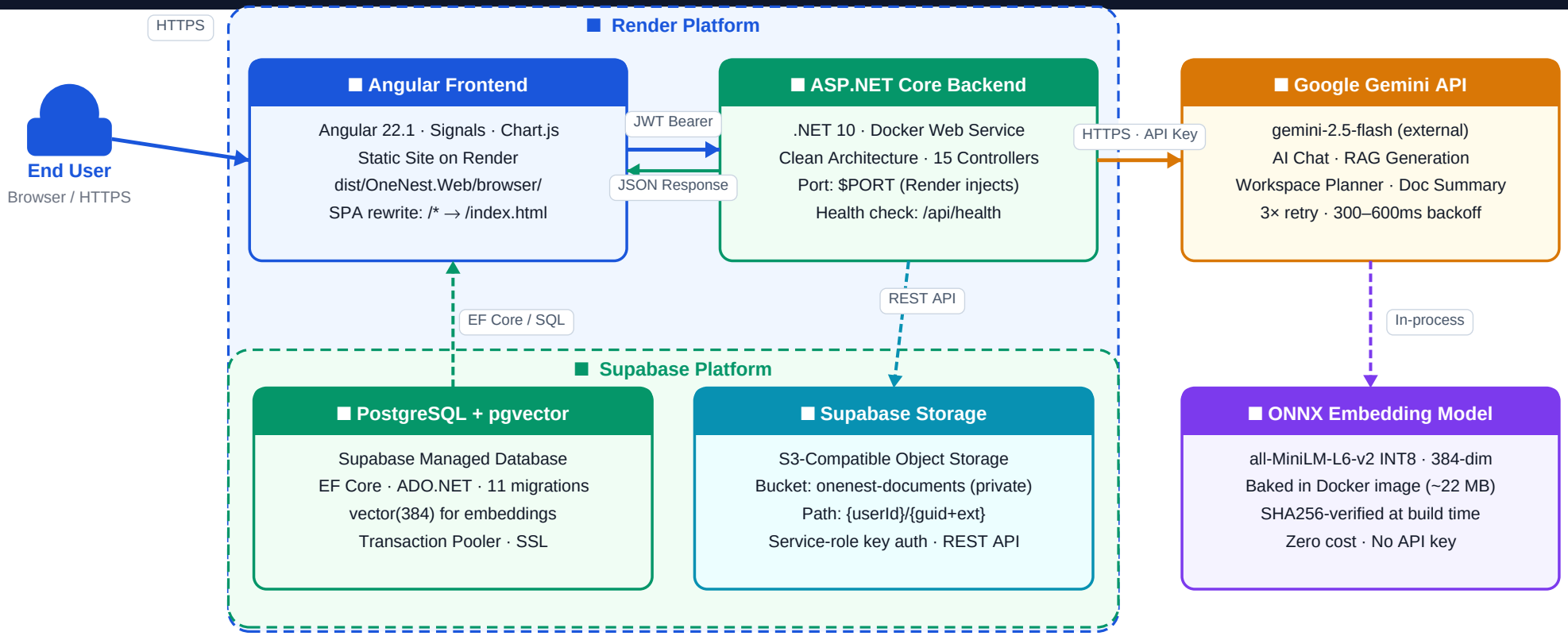
OneNest AI

Architecture & Flow Diagrams

v1.0 · 20 August 2026 · Kanha Gupta

DIAGRAM 1 — System Architecture & Deployment

Render platform · Supabase (PostgreSQL + Storage) · Google Gemini · Local ONNX embedding



All API requests carry Authorization: Bearer <JWT> | Every DB query scoped to authenticated user (UserId)

ONNX model runs in-process (no network call) | ForwardedHeaders middleware trusts Render TLS proxy

DIAGRAM 2 — Clean Architecture Layers

Dependency rule: outer layers depend on inner; Domain has zero external dependencies

API Layer

- AuthController, NotesController, DocumentsController
- AIController, SemanticSearchController, TasksController
- ExpensesController, HealthHubController, AdminController
- MedicinesController, AppointmentsController, ContactController
- SettingsController, MedicalReportsController, MedicalRecordsController
- Program.cs · ForwardedHeaders · CORS · JWT · Swagger

Infrastructure

- OneNestDbContext (EF Core) · 11 migrations
- EmbeddingRepository (raw ADO.NET · pgvector)
- SupabaseFileStorageService · LocalEmbeddingProvider
- GeminiProvider · RagService · SemanticSearchService
- TextChunker · BackfillService · SemanticIndexService
- AIConversationService · AIWorkspaceOrchestrator

Application

- IAuthService · IDocumentService · INoteService
- IRagService · ISemanticSearchService · ISemanticIndexService
- IEmbeddingProvider · IFileStorageService · IAIService
- All DTO records (Request / Response)

Domain

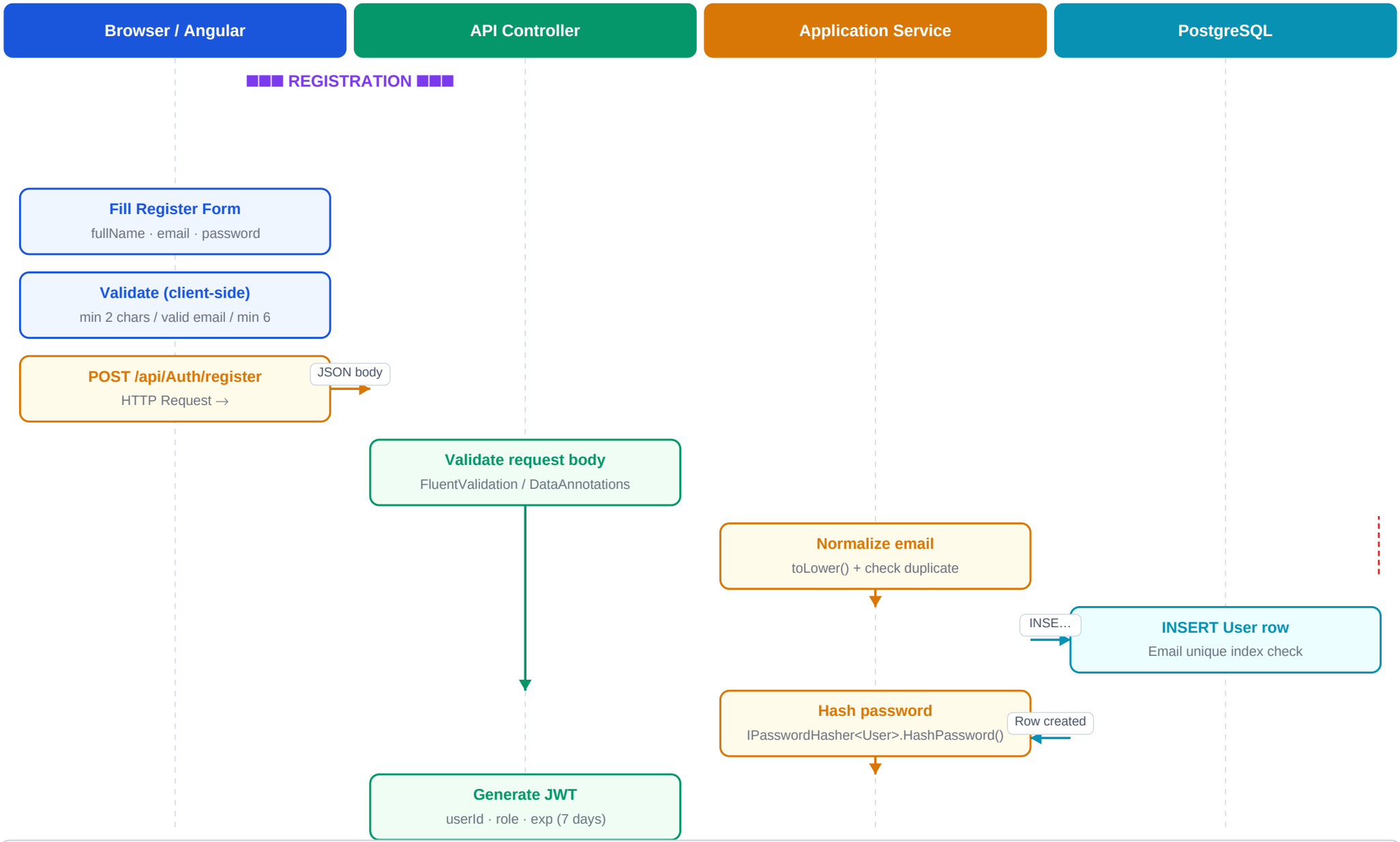
- User · Note · TaskItem · Expense · Document
- Medicine · Appointment · MedicalRecord · MedicalReport
- AIConversation · AIMessage · UserSettings · ContactMessage
- EmbeddingRecord | All Enumerations

Dependency Inversion: Infrastructure implements Application interfaces; registered via DI in Program.cs

EmbeddingRecords bypasses EF Core — managed with raw ADO.NET to use pgvector vector(384) column type directly

DIAGRAM 3 — Authentication & Session Flow

Registration · Login · JWT lifecycle · Client-side expiry guard · Account deletion



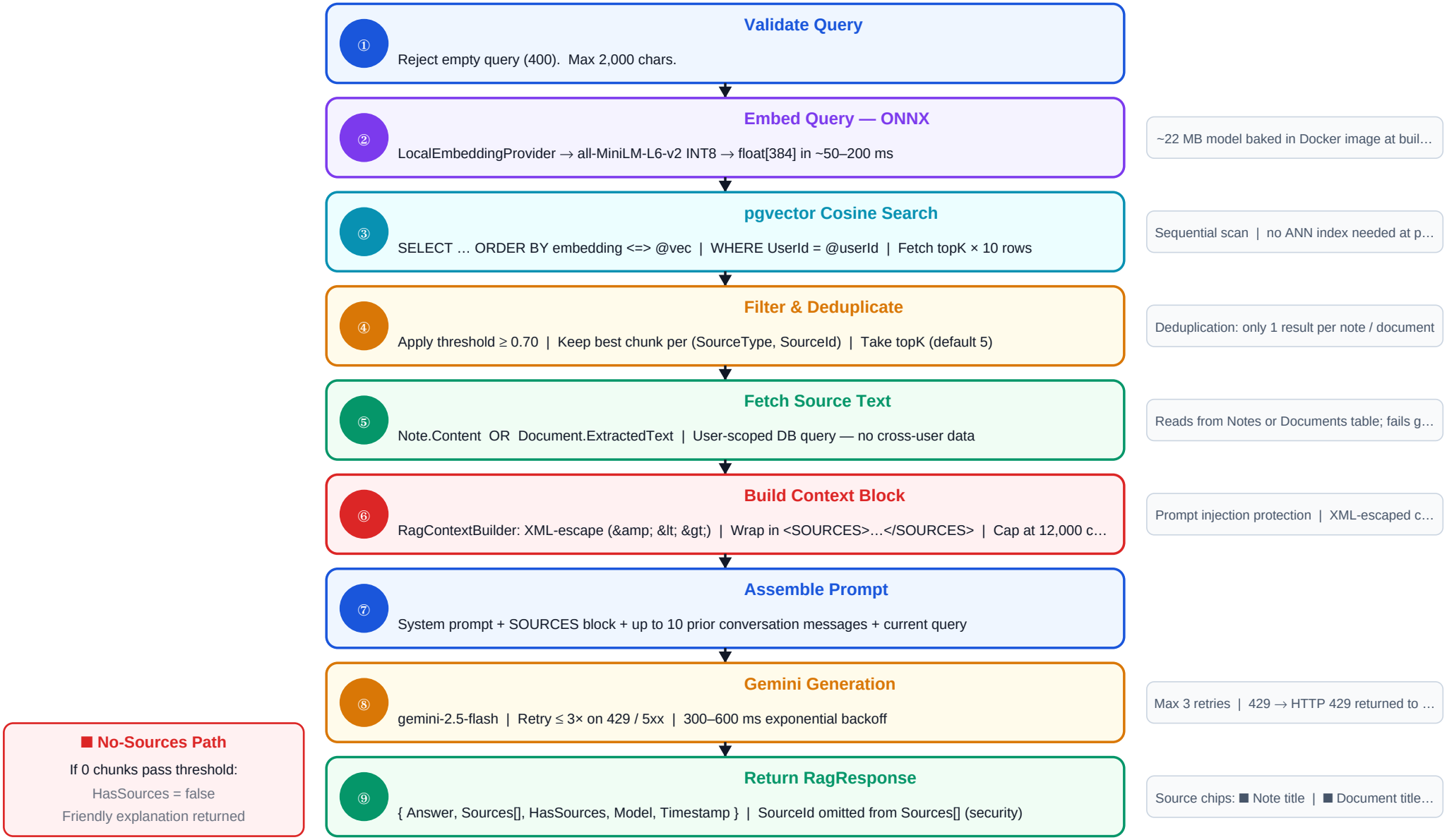
JWT payload: { userId (Guid), fullName, email, role ("User"|"Admin"), exp }

authGuard decodes JWT exp client-side — expired token → immediate redirect to /login, no API call made

Account deletion: POST /api/Auth/delete-account (password-confirmed) purges files + embeddings + all user data

DIAGRAM 4 — RAG Pipeline Flow (9 Steps)

POST /api/ai/rag: natural language query → grounded answer with source citations



Config (appsettings / env): TopK=5 · MaxTopK=20 · SimilarityThreshold=0.70 · MaxContextCharacters=12,000 · MaxConversationMessages=10

Embedding dimension: 384 (Local ONNX) | pgvector cosine metric: <=> | All queries strictly userId-scoped

DIAGRAM 5 — Semantic Indexing Flow

Auto-index on Note/Document save · Manual backfill · Fire-and-forget background Task.Run

■ Auto-Index: Note Save

1. User creates / updates Note
2. POST or PUT /api/Notes
3. NoteService saves entity to DB
4. HTTP 200/201 returned immediately
5. Task.Run() — new DI scope
6. Delete old chunks from EmbeddingRecords
7. TextChunker splits Title+Content
8. EmbedAsync() per chunk (ONNX)
9. Upsert EmbeddingRecord rows

■ Auto-Index: Document Upload

1. User uploads document (≤25 MB)
2. POST /api/Documents (multipart)
3. Validate file type + quota
4. Save file to Supabase Storage
5. Extract text (best-effort)
6. Persist Document entity (HTTP 201)
7. Task.Run() — new DI scope
8. Chunk ExtractedText → Embed → Upsert
9. Errors logged; never propagated

■ Manual Backfill

1. POST /api/semantic-search/backfill
2. → HTTP 202 Accepted immediately
3. Task.Run() in background
4. BackfillService.RunAsync()
5. Index ALL user notes
6. Index docs with ExtractedText
7. Skip docs without text
8. Per-item errors swallowed
9. Returns BackfillResult counts

Fire-and-forget: HTTP response always returned immediately; background Task.Run uses a new DI scope to avoid cancellation

Unique DB constraint on EmbeddingRecords: (UserId, SourceType, SourceId, ChunkIndex) — old chunks deleted before re-indexing

TextChunker: ChunkSize=1200 chars (~300 tokens) | Overlap=240 chars (~60 tokens) | Sentence-aware splitting

DIAGRAM 6 — AI Workspace Context Selection Flow

How the AI selects relevant personal data tools and injects them into the Gemini system prompt

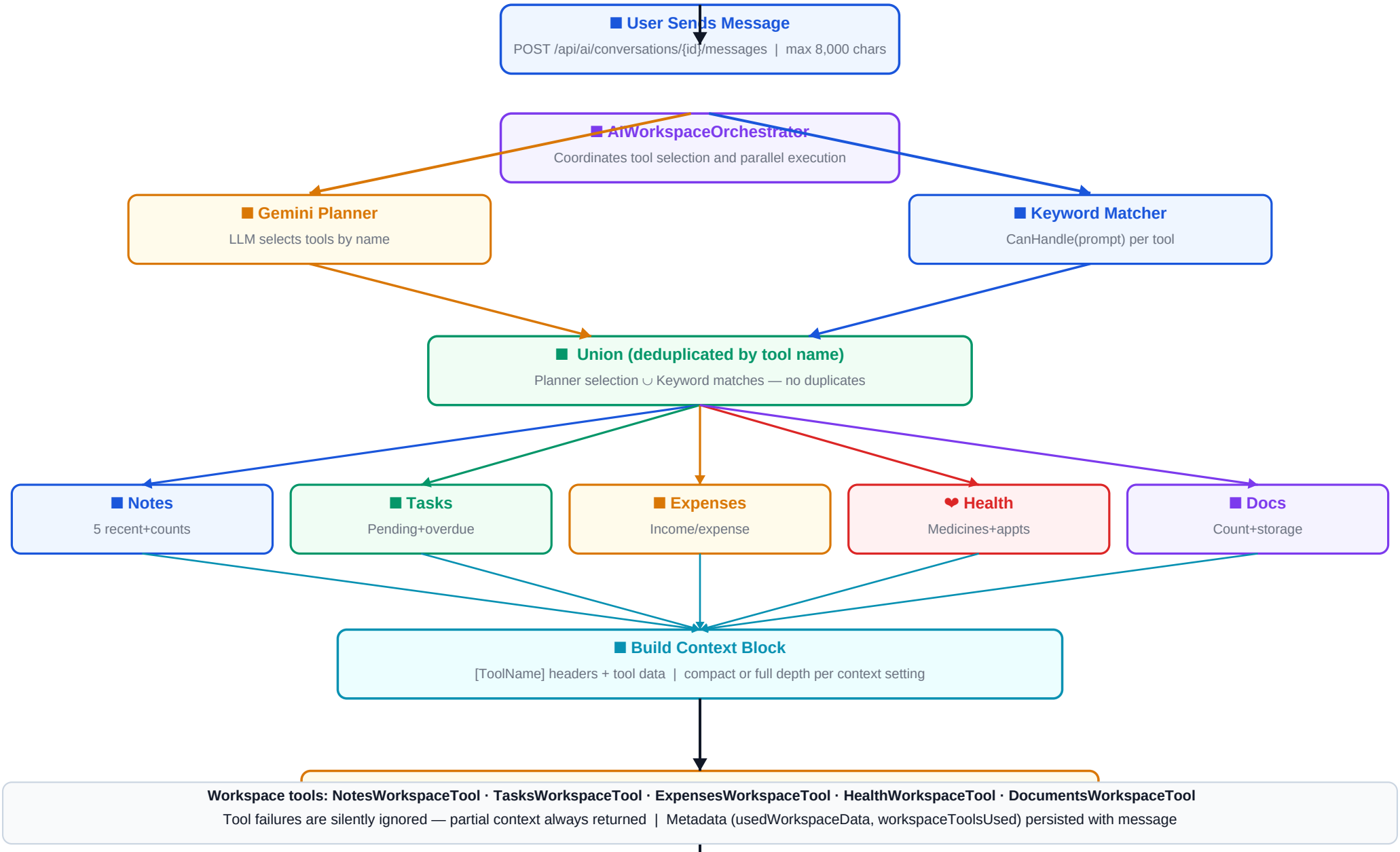
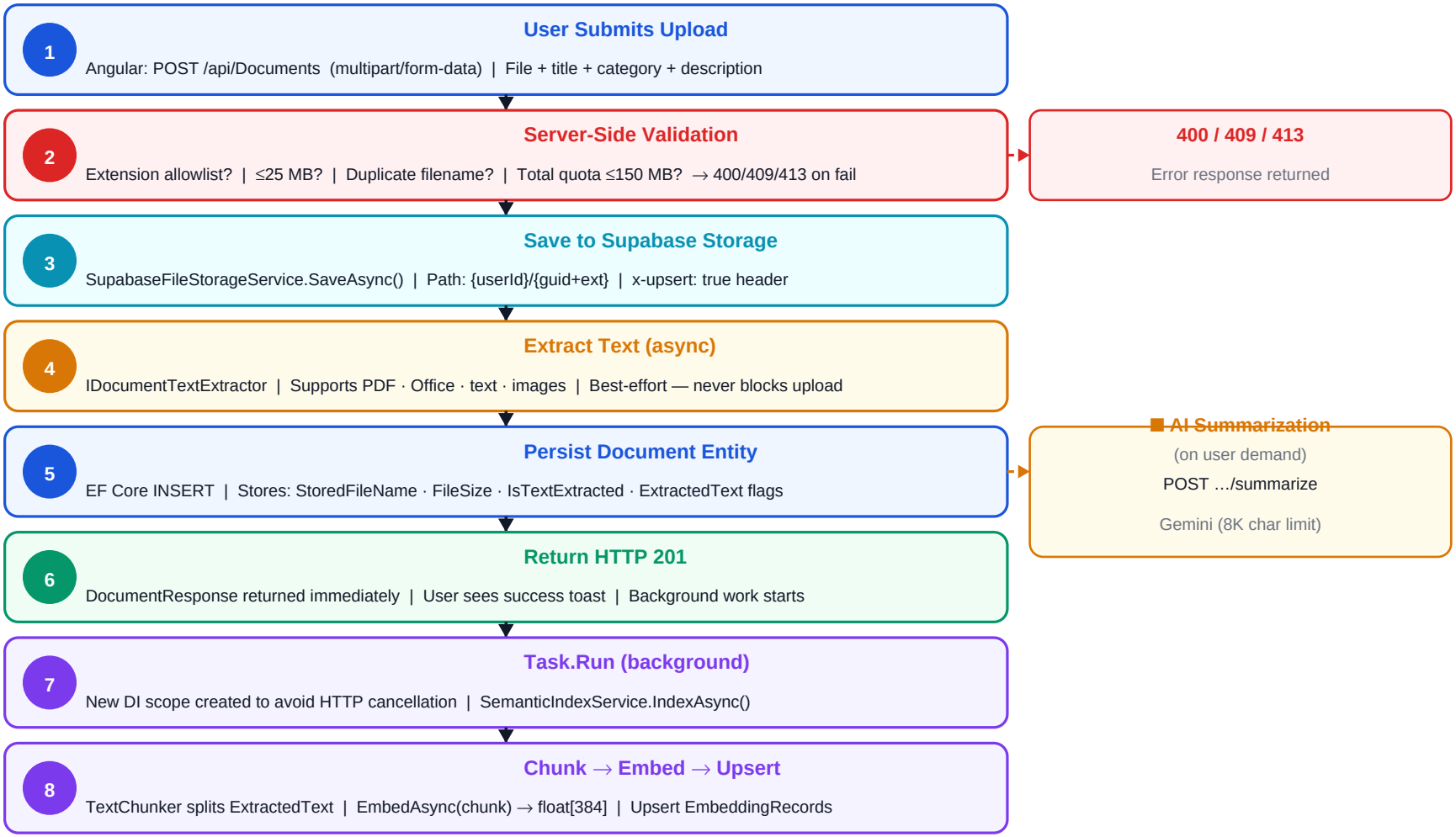


DIAGRAM 7 — Document Upload & Processing Flow

Upload → Validation → Supabase Storage → Text Extraction → HTTP 201 → Background Semantic Index



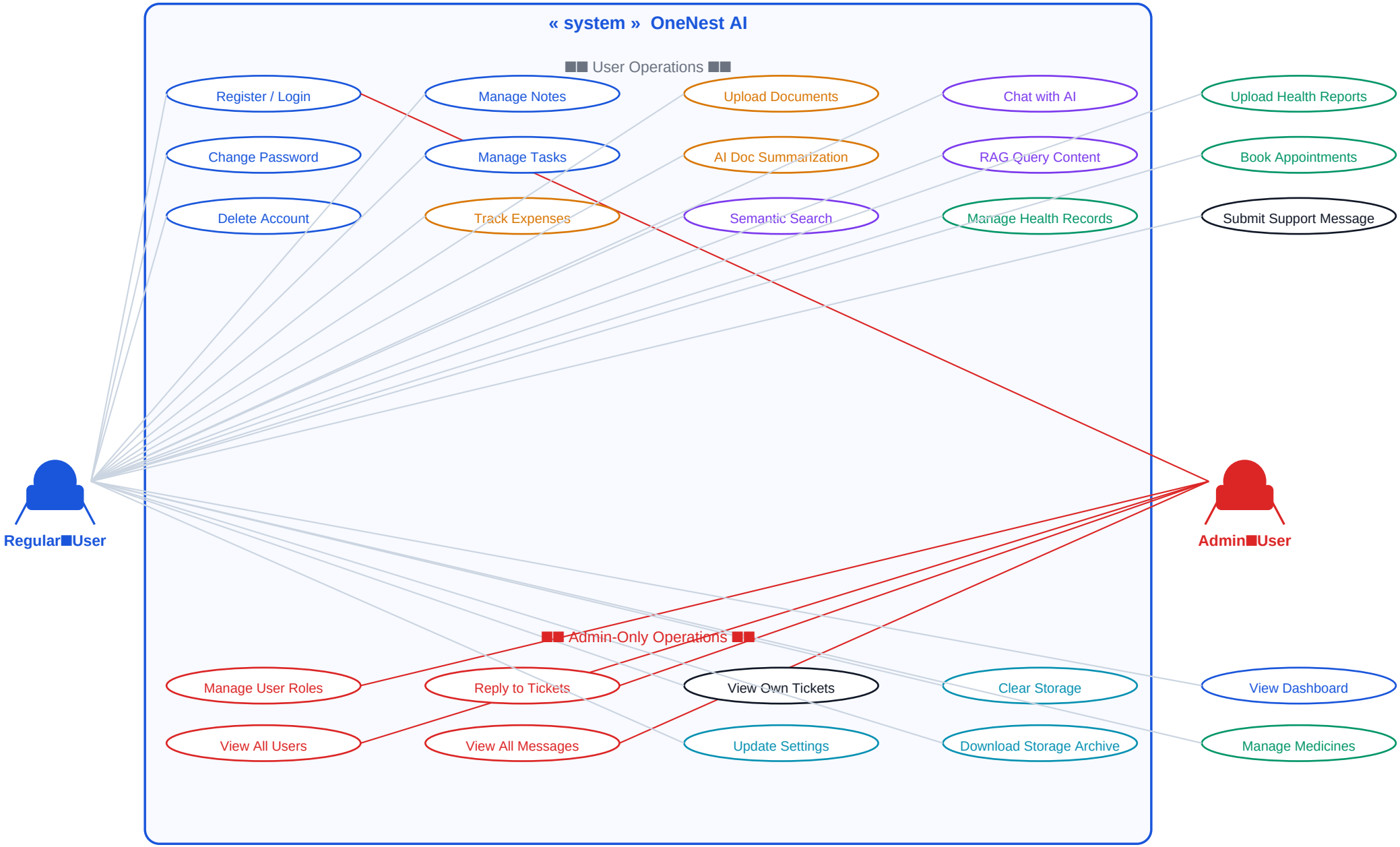
Allowed extensions: .pdf .doc/x .xls/x .ppt/x .txt .csv .rtf .jpg .jpeg .png .gif .bmp .webp

Storage limits: 25 MB per file | 150 MB total per user (documents + health reports combined)

Background indexing uses Task.Run + new DI scope — HTTP response is never delayed by ONNX inference

DIAGRAM 8 — Use Case Diagram

Regular User and Admin actor use cases within the OneNest AI system boundary

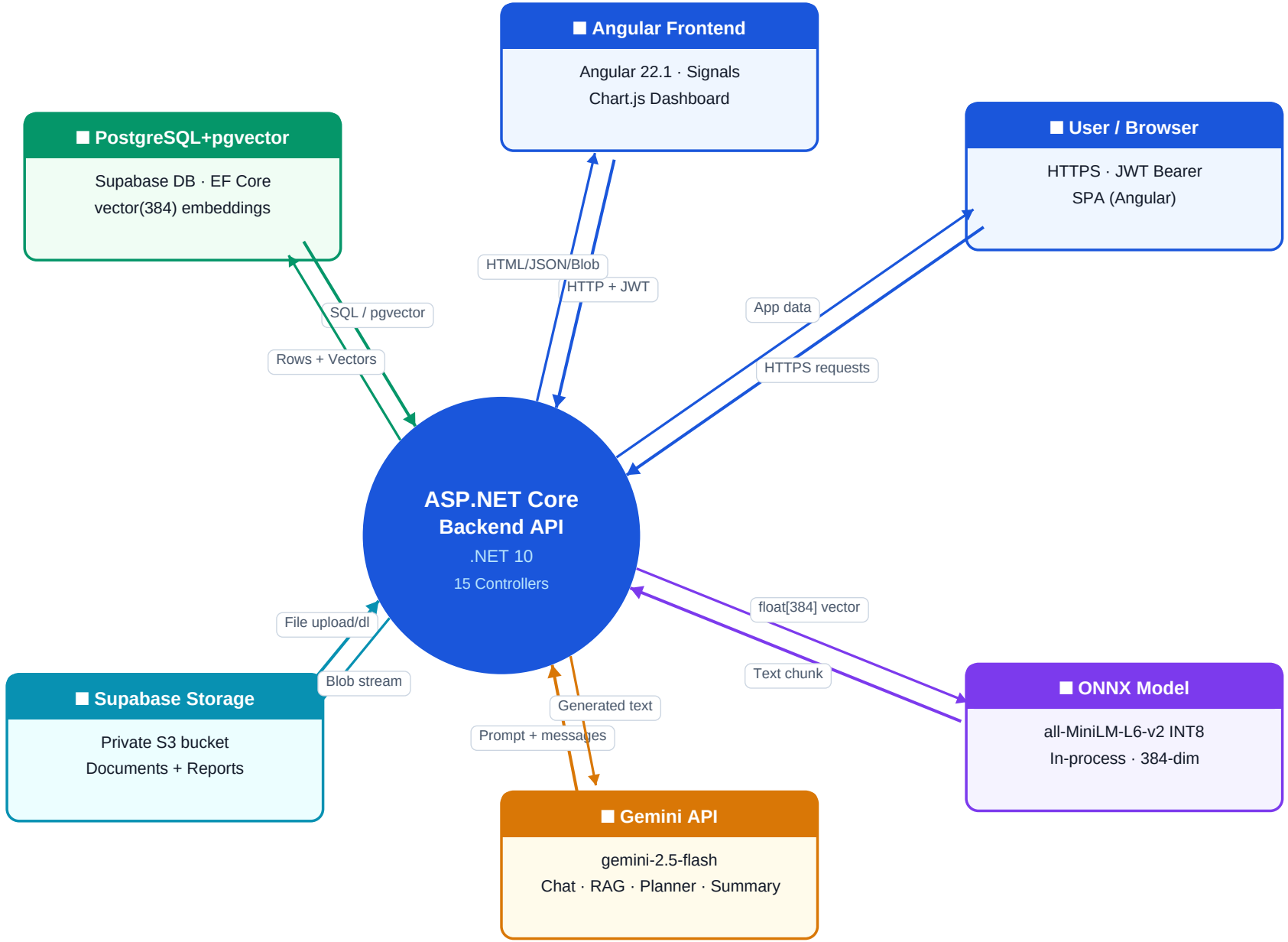


Admin guard: /admin route requires role="Admin" in JWT | [Authorize(Roles="Admin")] on AdminController

Admin also has access to all user operations | Admin cannot change their own role

DIAGRAM 9 — High-Level Data Flow Diagram

Data movement between User, Frontend, Backend API, Database, Storage, and External Services

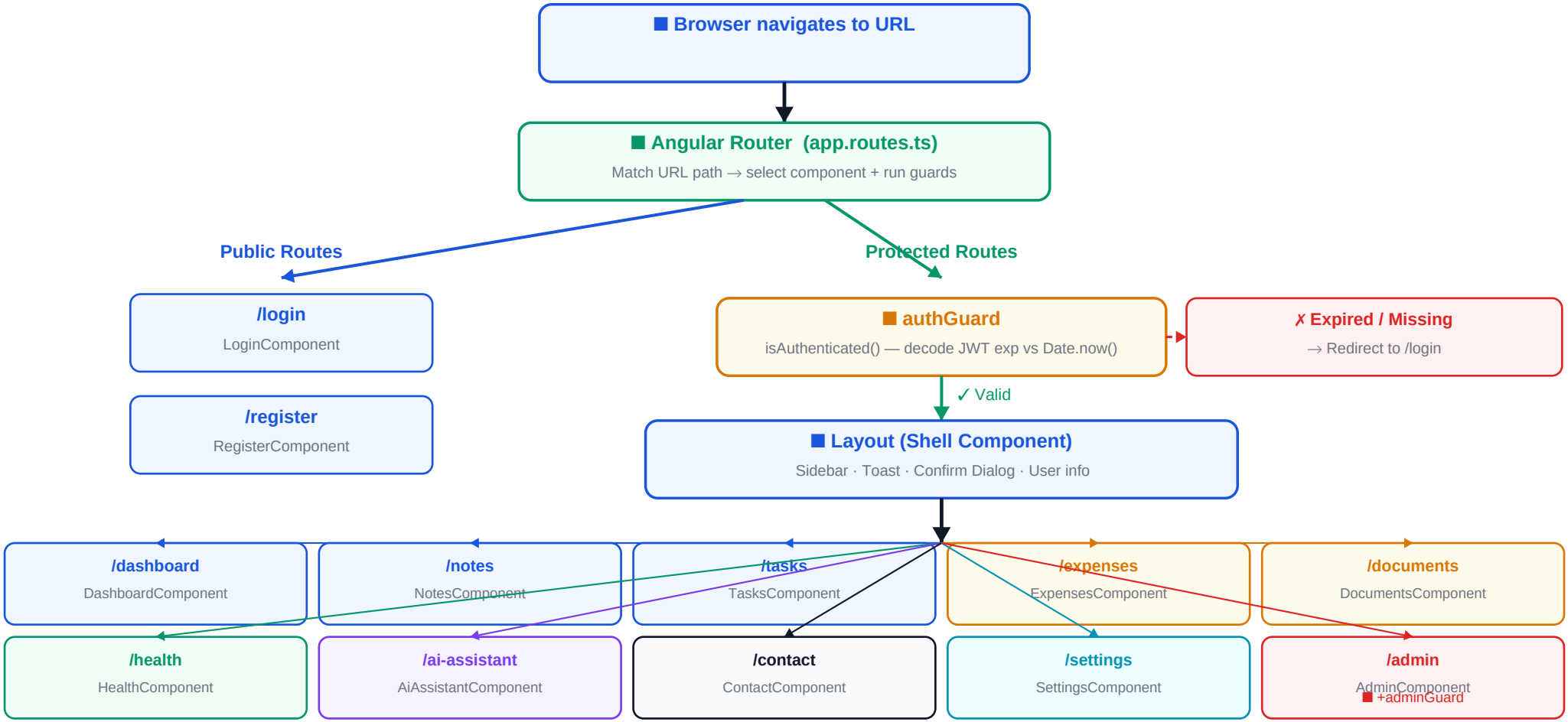


ONNX model runs in-process inside the Docker container — no network call, zero API cost for embeddings

Supabase Storage accessed only for file blobs (documents + health reports) | All other data in PostgreSQL via EF Core

DIAGRAM 10 — Frontend Navigation & Routing Flow

Angular Router · authGuard (JWT expiry) · adminGuard (role check) · Layout shell



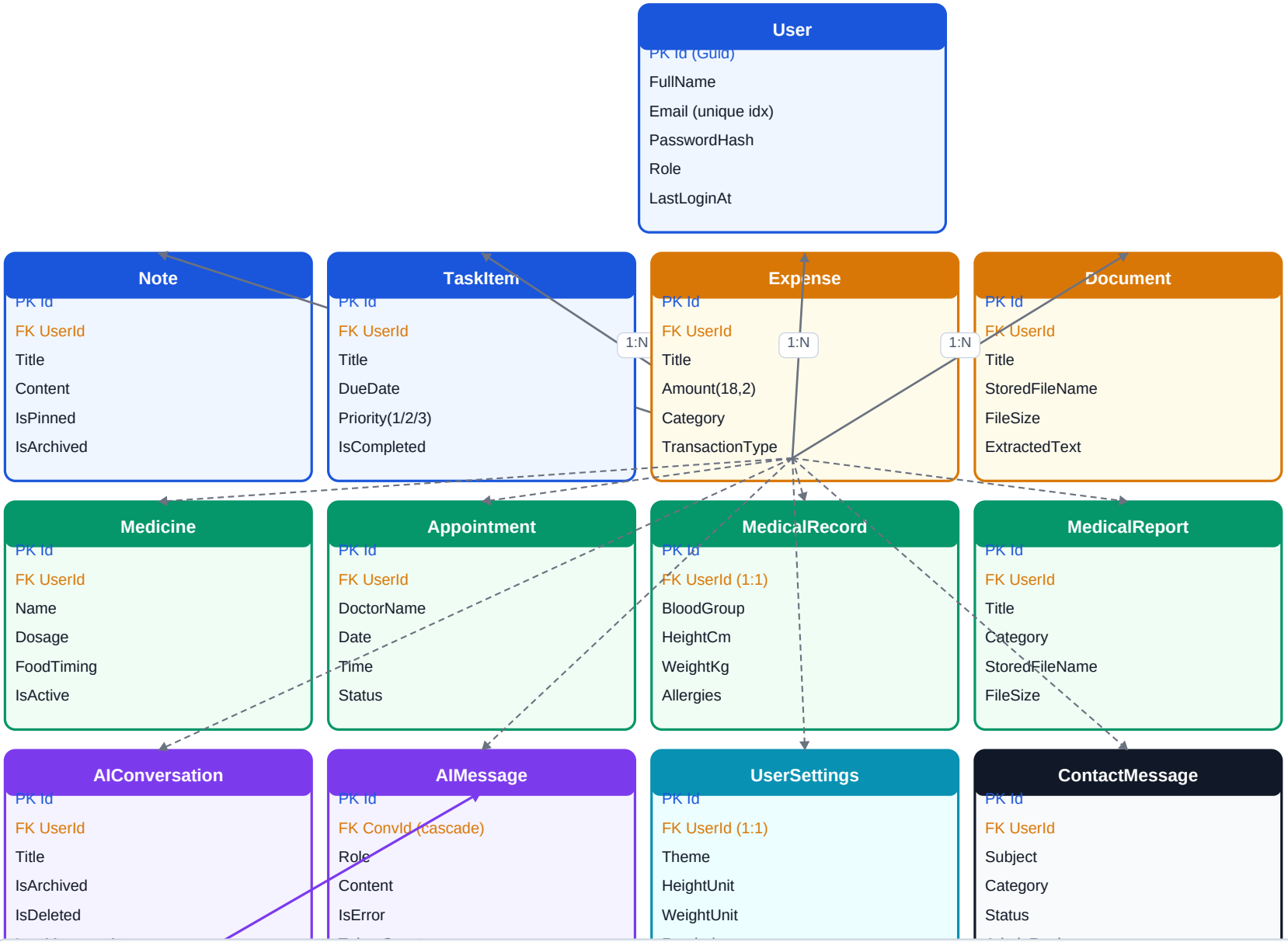
authInterceptor: auto-appends Authorization: Bearer <token> to every outgoing HTTP request

unauthorizedInterceptor: catches 401 responses globally → clears localStorage → redirects to /login

adminGuard: checks currentUser()?.role === "Admin" — redirects non-admins to /dashboard

DIAGRAM 11 — Database Entity Relationship Overview

Key entities, foreign keys, cardinalities, and pgvector EmbeddingRecords



PK = Primary Key | FK = Foreign Key | Every entity carries FK UserId for strict per-user isolation

EmbeddingRecords: NOT in EF Core — managed by raw ADO.NET | pgvector extension required | 1:1 with User for UserSettings

1:N cascade: deleting AIConversation also deletes all AIMessages | MedicalRecord is 1:1 per user (upsert semantics)

DIAGRAM 12 — End-to-End Request Lifecycle

Middleware stack · DI resolution · Controller → Service → Repository → DB and back

HTTP Request ↓

ForwardedHeaders
X-Forwarded-For/Proto
from Render TLS proxy

CORS Middleware
Env-driven allowed origins
Cors__AllowedOrigins__*

JWT Authentication
Validates Bearer token
Decodes userId + role

Authorization
[Authorize] / [Roles]
403 if insufficient

Route Matching
api/[controller]/[action]
Map to action method

Controller Action
ICurrentUserService.UserId
DTO binding + validation

Application Service
Business logic layer
Orchestrates operations

Repository / EF
SQL + pgvector queries
Map to domain entities

PostgreSQL
Execute query
Return result set

↑ HTTP Response

■ AI Chat Request

1. JWT → userId extracted
2. AIWorkspaceOrchestrator runs
3. Gemini Planner + keyword match
4. Selected tools execute
5. Context block assembled
6. Last 20 msgs from DB
7. GeminiProvider.GenerateAsync()
8. 3× retry on 429 / 5xx
9. Save AIMessage + metadata
10. Return ChatResponse

■ Document Upload

1. JWT → userId extracted
2. Extension + size validated
3. Quota check (150 MB total)
4. Duplicate filename check
5. Supabase Storage.SaveAsync()
6. Text extraction (async)
7. EF Core INSERT Document
8. HTTP 201 returned
9. Task.Run → index in BG
10. Chunk → Embed → Upsert

■ RAG Query

1. JWT → userId extracted
2. Query validated (non-empty)
3. ONNX EmbedAsync() → float[384]
4. pgvector cosine scan
5. Filter threshold ≥ 0.70
6. Dedup by (Type, SourceId)
7. Fetch Note/Doc text
8. Build XML-safe context
9. GeminiProvider.GenerateAsync()
10. Return RagResponse + Sources

■ Login Request

1. No JWT required
2. Normalize email (toLower)
3. SELECT user WHERE email
4. PasswordHasher.VerifyHash()
5. 401 if hash mismatch
6. UPDATE LastLoginAt
7. JwtTokenGenerator.Generate()
8. Return AuthResponse
9. Client stores to localStorage
10. Navigate to /dashboard

❤ Health Record

1. JWT → userId extracted
2. PUT /api/MedicalRecords
3. Find or create record
4. HeightCm / WeightKg in SI
5. (Unit conv. done in UI)
6. EF Core SaveChanges()
7. Return 200 + record

■ Admin Role Change

1. JWT → role = Admin
2. [Authorize(Roles="Admin")]
3. 403 for non-admin JWT
4. Cannot change own role
5. PATCH /api/Admin/users/{id}
6. EF Core UPDATE User.Role
7. Return AdminUserResponse